DATABASE SYSTEMS · REVIEW 3

DESIGN AND IMPLEMENTATION OF A COURIER DATABASE

Courier parcel booking and tracking management system

Student name and roll number:P. SAI RITHVIK YADAV - 25WU0102204

Department : CSE AIML

Subject: Database Management Systems · Faculty guide: DR. NAGARAJU

REVIEW 3

Courier parcel management

PROBLEM

Database Management Systems

Manual courier operations make every parcel harder to trust

MANUAL OPERATION

Records are fragmented

- Real-time parcel tracking is difficult.
- Booking, payment, and delivery records are error-prone.
- Branch and hub information is not centralized.
- Delayed parcels and failed deliveries are difficult to identify.

SYSTEM OBJECTIVE

One dependable operating record

- A relational database for courier operations.
- Accurate booking, routing, tracking, and payment records.
- Integrity through keys and constraints.
- Operational and revenue reports.
- Python application connected to MySQL.

The system connects a Python application to one shared courier record

Core relationships

Customer 1:N Booking; each Booking is associated with a Parcel.

Parcel 1:N Scan Event and 1:N Delivery Attempt.

Booking 1:N Charge and 1:N Payment.

Branch and Hub support parcel routing through Route records.

Architecture

User/Application Interface → Python Backend → MySQL Relational Database → SQL Reports.

ER model entities

Customer, Branch, Hub, Service Type, Parcel, Booking, Route, Scan Event, Vehicle, Delivery Agent, Delivery Attempt, Charge, and Payment.

REVIEW 3

Courier parcel management

DATABASE DESIGN

From ER model to relational tables

Normalization turns the ER model into dependable relational tables

TABLE	REPRESENTATIVE COLUMNS	KEY CONTROL	PURPOSE
CUSTOMER	CustomerID, Name, Phone, Email	PK · Phone UNIQUE	Customer identity
BRANCH	BranchID, BranchName, Location	PK	Network location
PARCEL	ParcelID, TrackingNumber, Weight, BookingID	PK · UNIQUE · FK	Parcel record
BOOKING	BookingID, CustomerID, BookingDate, ServiceTypeID	PK · FK	Booking transaction
SCAN_EVENT	ScanID, ParcelID, BranchID, ScanTime, Status	PK · FK	Tracking history
PAYMENT	PaymentID, BookingID, Amount, PaymentMode, PaymentStatus	PK · FK	Payment record

NORMALIZATION WORKFLOW

1 **1NF**

Atomic values and no repeating groups.

2 **2NF**

No partial dependency on a composite key.

3 **3NF**

No unnecessary transitive dependencies.

Constraints include PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, and DEFAULT. The complete implementation includes all required entities.

REVIEW 3

Courier parcel management

SQL OPERATIONS

MySQL operations support booking, tracking, and reporting

TRACKING QUERY

```
SELECT p.TrackingNumber, s.Status,
s.ScanTime
FROM Parcel p JOIN Scan_Event s ON
p.ParcelID = s.ParcelID
WHERE p.TrackingNumber = 'TRK1001'
ORDER BY s.ScanTime DESC;
```

OPERATIONS SUPPORTED

INSERT · add a customer or create a parcel booking. **SELECT** · search the latest event by tracking number.

JOIN · combine parcel and customer details. **GROUP BY** · calculate branch-wise booking totals.

UPDATE · change a parcel tracking status. **VIEW** · create a reusable parcel tracking report.

TRACKINGNUMBER	STATUS	SCANTIME
TRK1001	In Transit	Scan time recorded at the latest branch event

REVIEW 3

Courier parcel management

APPLICATION LAYER

The Python application turns database records into a courier workflow

MAIN MODULES

- Parcel booking with customer and service selection; charge calculation by weight, service type, and rules; payment recording.
- Tracking search by unique tracking number; scan event and route movement updates.
- Delivery agent assignment and confirmation; failed delivery attempt recording; report generation.

ILLUSTRATIVE DASHBOARD MOCKUP

Courier Operations · ILLUSTRATIVE

Tracking search	TRK1001
Parcel status	In Transit
Booking summary	Confirmed
Recent scan history	Branch received · Route assigned · Hub departure

Workflow: Book Parcel → Calculate Charges → Generate Tracking Number → Record Scans → Assign Delivery Agent → Deliver Parcel → Generate Report.

Python turns records into action

REVIEW 3

Courier parcel management

VALIDATION

Rules, reports, and test evidence

Business rules make invalid courier states difficult to create

BUSINESS RULES

- Tracking numbers are unique; parcel weight and charges are positive.
- Mandatory fields cannot be NULL; foreign keys must reference valid records.
- A delivered parcel cannot receive subsequent ordinary transit scans.
- Payment cannot exceed the permitted booking balance.
- Status transitions follow valid order and chronological scan times.

REPORTS

Current location and status, delayed shipments, branch workload, route performance, failed delivery attempts, and revenue/payment summaries.

TESTING MATRIX

TEST CASE	INPUT / ACTION	EXPECTED RESULT	ACTUAL RESULT
Duplicate tracking number	Insert existing TrackingNumber	Reject duplicate value	[Enter actual result]
Negative weight	Insert Weight < 0	Reject invalid weight	[Enter actual result]
Invalid customer ID	Book with unknown CustomerID	Reject foreign-key reference	[Enter actual result]
Successful booking	Valid customer, parcel, and service	Create booking and tracking record	[Enter actual result]
Scan after delivery	Add ordinary transit scan	Reject invalid state transition	[Enter actual result]

Actual Result and Status must be populated from the executed test record.



REVIEW 3 · CONCLUSION

The project creates a foundation for more accurate courier operations

Results

- Relational design for booking, routing, tracking, and payment; structured storage with SQL search and reporting.
- Integrity through constraints and validation; Python integration for the courier workflow.

Limitations

[Insert incomplete modules, limited sample data, or remaining integration gaps if applicable].

Future scope

- Live GPS vehicle tracking; SMS or email status notifications.
- Online payment gateway integration; customer and administrator login roles.
- Advanced delay analytics and route optimization.

The system aims to improve tracking accuracy, data consistency, and operational efficiency.

Thank You